AI Bootcamp Week 3: Model Invocation, Adversarial Agents, and Bounded Agents

© 2026 Improving

TIMING

- **Monday:** 1:00 PM – 5:00 PM (travel morning: 4 hours)
- **Tuesday–Thursday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each)
- **Friday:** 9:00 AM – 12:00 PM (travel afternoon: 3 hours)

© 2026 Improving

OBJECTIVES

By end of Week 3, the candidate can:

- Explain the DLP automation spectrum and what evidence is required to move up each step
- Explain why the DLP defaults to skills with `disable-model-invocation: true`, and articulate the three graduation criteria for enabling model invocation
- Write invocation test cases (positive trigger, negative trigger, quality) and use them to make a documented enable-or-gap decision on a real skill
- Build an adversarial agent that challenges a skill's output, and explain how it complements (not duplicates) a Python evaluation script
- Run an evaluator-vs-adversarial comparison and use the delta to improve either the EC or the skill
- Compose skills into a bounded agent: a multi-step pipeline run in one IDE session with evaluation and adversarial review at key steps
- Explain context window degradation, why bounded agents hit a ceiling, and what a custom harness is (awareness level; building one is Stage 4 work)
- Describe the IIOS stage framework: where DLP Completion sits (Stage 3, Task) and what Stage 4 (Workflow) requires

© 2026 Improving

THE CORE CONCEPT: THE AUTOMATION SPECTRUM

Weeks 1 and 2 were entirely about skills with `disable-model-invocation: true`: skills you invoke explicitly, at a time you choose, with output you review before anything downstream happens. This week introduces the rest of the automation spectrum:

```
Skills (invocation disabled)  : you invoke explicitly, full control
  ↓
Skills (invocation enabled)   : the model decides when to invoke
  ↓
Adversarial Agents            : AI challenges AI output
  ↓
Bounded Agents                : multi-step orchestration in the IDE
  ↓
Custom Harnesses              : programmatic orchestration (Stage 4, demoed not built)
```

The key principle: Each step extends what you can trust AI to do. Each step requires earning the right to the next one through evaluation evidence. You do not enable model invocation on a skill that has not been through documented EDD iteration cycles. You do not chain skills into a bounded agent before each skill passes its own evaluation. You do not build a custom harness on weak Stage 3 foundations; that produces a slop cannon, where errors compound through every connected step.

A note on vocabulary: earlier versions of this program taught deterministic hooks and sentinel files as the governance mechanism between automated steps. That content is retired from the DLP. Adversarial agents and Python evaluation scripts cover quality assurance in-program; deterministic hooks and sentinel files still exist, but as Stage 4 infrastructure for automated gates, which is beyond this program's scope.

© 2026 Improving

MONDAY (1:00 PM – 5:00 PM): THE SPECTRUM AND THE INVOCATION DECISION

1:00 – 2:30 PM: INVOCATION DISABLED VS. ENABLED

Every skill you built in Weeks 1 and 2 has `disable-model-invocation: true` in its frontmatter. You run it via slash command, at a time you choose. This is the default mode during a DLP engagement, and it is a deliberate design decision, not a limitation.

Why the DLP defaults to invocation-disabled:

A skill with invocation disabled only runs when you run it. If the output is wrong, only you are affected. You learn, you iterate, you improve it. That is the EDD discipline.

A skill with invocation enabled runs when the model decides to run it. If the model invokes the skill at the wrong time, it may produce output that downstream processes act on before anyone reviews it. And because different people on a team may have different expectations about when the skill should trigger, premature auto-invocation creates invisible disagreement about workflow that surfaces as confusing or inconsistent outputs.

Reconnect to Prompt/Harness/Model: the harness controls invocation, not the model. `disable-model-invocation: true` tells the harness "only run this skill when a human explicitly invokes it." Flipping the flag tells the harness "the model may decide to run this skill based on context." The model still cannot do anything the harness does not allow; you are changing what the harness allows.

The graduation frame: enabling model invocation is not a conversion or a migration. It is a configuration decision you earn with evidence. The old question was "should this be a skill or a command?" Everything is already a skill. The only question is "has this skill earned auto-invocation?"

Exercise: Review two of your Week 1 or Week 2 skills. For each, answer in writing:

- Could this skill be invoked at the wrong time and cause harm (wrong output, downstream dependency failures)?
- Does the right invocation time depend on things the model might judge differently from a human?
- If two team members had different opinions about when this skill should run, what would break?

2:30 – 4:00 PM: THE THREE GRADUATION CRITERIA

A skill is ready for model invocation when the team has agreed, in writing, on:

1. **Exact trigger inputs.** What specific input conditions should cause the model to invoke this skill? Not "when it seems relevant." Specific, describable input conditions. Your evaluation history tells you exactly which inputs produce good output; those are your candidate trigger conditions.
2. **Output specification.** What does the skill do when triggered? What output does it produce? This must be specific enough that you can verify the skill triggered correctly.

© 2026 Improving

3. **At least 3 verified trigger test cases.** At least 3 specific scenarios where the skill SHOULD invoke, verified to confirm that (a) the model actually does invoke it when those conditions are present, and (b) the output meets the specification from criterion 2. If you cannot name three specific scenarios and verify both invocation and output quality, you do not understand the invocation conditions well enough to automate it.

The most important criterion is #3. The difficulty of naming three verifiable trigger scenarios is the signal. If it is hard, you need more evaluation, not less.

The EDD prerequisite: a skill should have at least 3 documented iteration cycles before model invocation is enabled. If it does not have that history, the prerequisite is more EDD work, not graduation.

Exercise: Take the two skills from the previous exercise. For each, attempt to write all three graduation criteria in full. If you cannot write a criterion, write down specifically what is missing. If you cannot write it, you do not have consensus; you have an opinion.

4:00 – 5:00 PM: OVER-INVOCATION AND UNDER-INVOCATION

When skills go wrong with invocation, there are two failure modes:

- **Under-invocation:** the skill does not run when it should. A human does the work manually. Annoying, recoverable.
- **Over-invocation:** the skill runs when it should not. Output gets produced, and may get committed or acted on, before anyone notices. This is the most common and most urgent invocation failure in the field.

Key insight: Over-invocation is more dangerous than under-invocation. An un-invoked skill means a human does the work. An over-invoked skill means a machine did work no one expected, and the evidence may already be in the codebase.

Defense: negative invocation test cases. Scenarios where the skill should NOT invoke, verified to confirm it stays silent. Every over-invocation incident in the field becomes a new negative test case and a tightened invocation boundary.

© 2026 Improving

TUESDAY (9:00 AM – 5:00 PM): ENABLING MODEL INVOCATION

9:00 – 10:30 AM: EDD FOR INVOCATION: THE THREE CASE TYPES

Skills with invocation enabled require a different kind of test than output quality alone. You need all three case types:

- **Positive trigger cases:** input scenarios where the skill SHOULD invoke. Expected: the model invokes it, and the output meets the specification.
- **Negative trigger cases:** input scenarios where the skill SHOULD NOT invoke. Expected: the model handles the input another way and the skill stays silent.
- **Quality cases:** given that the skill did invoke, does the output meet the EC you have been evaluating all along?

Target: 3 verified positive trigger cases + 2 quality cases, plus negative cases for the boundaries you are least sure about. Write the test file before flipping any flag.

10:30 AM – 12:00 PM: THE GRADUATION ATTEMPT

Pick your most-iterated skill: the one with the most evaluation history from Weeks 1 and 2. Apply the full graduation check:

1. Write the three graduation criteria (trigger inputs, output specification, 3 verified trigger test cases)
2. Confirm the EDD prerequisite: 3+ documented iteration cycles
3. Write the invocation test file: positive, negative, quality cases
4. Flip `disable-model-invocation` to `false` and verify: set up each trigger context, confirm the model actually invokes the skill, check the output against spec. Then run the negative cases and confirm it stays silent.

12:00 – 1:00 PM: LUNCH

1:00 – 3:00 PM: BOTH OUTCOMES ARE VALID

You leave today with one of two artifacts:

- **Enabled:** the skill has model invocation enabled, with the graduation criteria and invocation test cases committed alongside it. Note the test cases so they can be re-validated later.
- **Gap document:** the skill stays at `disable-model-invocation: true`, and you have committed a written specification of exactly which criterion is not met and what evidence would close the gap.

Both are passing outcomes. What is not acceptable: vague "it is not ready" with no specific gap named. An unnamed gap means you have not looked.

If you enabled invocation, treat the rest of the week as a validation period. Did the skill invoke when it should? Did it stay silent when it should not? Address any over-invocation immediately, before it becomes a habit.

© 2026 Improving

3:00 – 5:00 PM: BOUNDARY AMBIGUITY WORKSHOP

In pairs, examine skill configurations where the invocation boundary is genuinely ambiguous. For each:

- Where would the model likely disagree with a human about whether to invoke?
- Is the failure mode over-invocation or under-invocation?
- What change to the skill's description or frontmatter would reduce the ambiguity?

Then simulate the team scenario: two engineers have independently enabled invocation on their own versions of the same skill, and the versions disagree about when to fire. Walk the alignment process: which trigger test cases does each version pass and fail on the other's test file? This is why the DLP treats invocation as a team-level, evidence-backed decision rather than an individual convenience.

© 2026 Improving

WEDNESDAY (9:00 AM – 5:00 PM): ADVERSARIAL AGENTS

9:00 – 10:30 AM: AI CHALLENGES AI

Your Python evaluation scripts measure output against the EC you wrote. That is retrospective quality: does the output meet the defined bar? But what about problems your EC did not anticipate?

An **adversarial agent** is a prompt that receives the output of another skill and attempts to find problems with it: missing requirements, logical errors, edge cases the primary skill missed, quality gaps the evaluation script did not catch.

The complementary pairing:

- **Evaluator:** measures against EC. "Does this output meet the defined quality bar?" Retrospective.
- **Adversarial agent:** challenges without EC. "What is wrong with this output that the EC did not catch?" Proactive.

The evaluator gives you a scorecard. The adversarial agent gives you a red team. Both are needed. A skill that passes its evaluator but fails its adversarial agent has a gap in its EC, which is itself a finding.

Why adversarial agents are AI-native, not code-native: an adversarial agent is a prompt. Every role can write one: product, process, testing, not just developers. This makes proactive quality challenge accessible to the whole team.

Adversarial agents are one form of verification, not the only form. Deterministic checks in your Python evaluation script handle structure: schema validation, field presence, format patterns. Adversarial agents check meaning: is the content actually correct, is the reasoning sound, what did the criteria miss. A complete quality strategy uses both.

10:30 AM – 12:00 PM: THE TWO-PROMPT PROBLEM

When you adopt an adversarial agent, you introduce a second prompt into your quality pipeline:

1. **The worker prompt** (your skill): produces the output. Optimized for producing good work.
2. **The evaluator prompt** (the adversarial agent): checks the output. Optimized for finding problems.

Both can have bugs. Both need evaluation. The mindset for "prompt to check the work" differs from "prompt to do the work": different persona, different EC, different tuning. An adversarial agent that produces false positives is just as harmful as one that misses real issues; it wastes time and erodes trust in the quality process. Common failures: agents that are too permissive ("looks good") or too aggressive (every output has "problems" that are not real).

What makes a good adversarial prompt:

- **Specific role:** "You are a senior QA engineer reviewing this output." Choose the reviewer role that is the natural critic of the skill's output. The best adversarial agent comes from the perspective the skill's author is least likely to have.
- **Clear mission:** "Find as many problems as you can. For each, explain why it is a problem."
- **Full output as input:** the adversarial agent sees everything the primary skill produced.
- **Structured findings:** each problem listed with what is wrong, why it matters, and a recommended fix.

© 2026 Improving

- **Not shown the original EC:** the adversarial agent does not know the criteria. It looks for anything wrong.
- **Calibrated severity:** every finding labeled blocker, major, minor, or nit. An agent that labels everything a blocker needs tuning.
- **Actionable findings:** specific recommendations, not "this is wrong." Vague findings are false positives in disguise.

12:00 – 1:00 PM: LUNCH

1:00 – 3:00 PM: BUILD AND RUN AN ADVERSARIAL AGENT

1. Choose one of your Week 2 SDLC skills: ideally the one you have been evaluating with a Python evaluation script.
2. Write an adversarial prompt for it: specific reviewer role, clear mission, full output as input, structured findings with calibrated severity.
3. Run it against 2–3 real outputs from that skill.
4. For each finding, classify it:
5. Real problem the EC did not cover → gap in your EC. Add a corresponding criterion to your evaluation script.
6. Real problem the EC covers but the skill violated → gap in the skill. Fix the skill.
7. False positive → tune the adversarial prompt.

Commit the adversarial prompt and its findings. The adversarial prompt is an artifact, a reusable quality tool that complements your evaluation script.

3:00 – 5:00 PM: THE EVALUATOR-VS-ADVERSARIAL COMPARISON

Close the loop. Run your evaluation script and your adversarial agent against the same 2–3 outputs. Document the comparison:

- What did the evaluator catch?
- What did the adversarial agent catch that the evaluator missed?
- What did the evaluator catch that the adversarial agent missed?
- What does the delta tell you about gaps in your EC?

Commit the comparison document. Most candidates find their EC cover the obvious quality dimensions but miss domain-specific risks that a well-chosen reviewer role catches. That delta is the value.

The value test: if your adversarial agent only finds what your evaluation script already catches, it is adding nothing. Refine the role, the mission, or the severity calibration until it finds things you did not anticipate.

© 2026 Improving

THURSDAY (9:00 AM – 5:00 PM): BOUNDED AGENTS AND WHAT COMES AFTER

9:00 – 10:30 AM: WHAT A BOUNDED AGENT IS

Everything you have built so far is single-step: one skill invocation, one model call, one output, human review. A **bounded agent** chains those steps into a multi-step workflow that runs within the IDE's built-in harness. You invoke a skill, review the output, invoke the next skill, review that output, and so on. The context window carries artifacts between steps: the stories are in context when you invoke `plan-story`, and the plan is in context when you invoke `implement-story`.

This is not a new tool. It is not a Python program or API calls. It is you, in your IDE, running skills you already built, in sequence, with your evaluation script and adversarial agent applied at key steps.

What a bounded agent can do:

- Run the full SDLC pipeline: `create-epics-stories` → `plan-story` → `implement-story` → `review-implementation`, in one session
- Run the evaluation script after each step (you invoke it manually, but it runs)
- Run the adversarial agent after key steps
- Produce a final artifact that includes all step outputs, evaluation scores, and adversarial findings

What a bounded agent cannot do:

- Give each step its own clean context. The context window accumulates, and output quality degrades at later steps.
- Evaluate automatically. You run the evaluation script manually at each step.
- Handle failures programmatically. If a step fails evaluation, you decide what to do, not the harness.
- Run unattended. You are in the loop at every step.

A bounded agent is a manually orchestrated chain with checkpoints, not automation. Treating it as automation is the failure mode. The human at each checkpoint is the governance.

10:30 AM – 12:00 PM: BUILD A BOUNDED AGENT

Run the full pipeline in a single IDE session, starting from a fresh session and a single story or feature request:

1. Invoke `create-epics-stories` (or start from `create-prd` if you have the full Week 2 chain working). Review the output.
2. Run the evaluation script against the output. Note the EC scores.
3. Invoke `plan-story`. Review the plan. Evaluate it.
4. Invoke `implement-story`. Review the code.
5. Run your adversarial agent against the code output. Note findings.
6. Invoke `review-implementation`. Review the review.

© 2026 Improving

7. Save the full session output as your bounded agent artifact.

Done when: you have run the chain end to end in one session, with evaluation and adversarial review at key steps, and you can point to where output quality changed at later steps.

12:00 – 1:00 PM: LUNCH

1:00 – 2:00 PM: CONTEXT WINDOW DISCIPLINE (FEEL IT FIRST, THEN NAME IT)

You just ran the full pipeline. You likely noticed: later steps produced output that was less sharp than earlier ones. That is not a model problem. It is a context window problem.

The degradation pattern you just experienced:

- Step 1 had a clean context window: good output
- Step 2 had Step 1's output in context: still good, slightly constrained
- Step 3 had Steps 1 and 2 in context: noticeable quality difference in complex cases
- Step 4 had everything in context: often where quality falls off in complex inputs

The model's attention spreads across the entire accumulated context. The IDE harness does not manage this for you; the window just grows.

What you can do about it within the IDE:

- Keep each step's output compact. Verbose output crowds the next step.
- Evaluate at each step. Catch degradation early, before it compounds.
- If a step degrades, start a fresh session with only the prior artifact pasted in: a manual clean-context workaround.
- Accept that the bounded agent has a ceiling. For long pipelines, the window fills and quality degrades.

2:00 – 3:00 PM: THE CUSTOM HARNESS (AWARENESS ONLY)

The solution to the bounded agent's ceiling is a **custom harness**: a Python program that orchestrates multiple model calls with clean context per step, passes artifacts between steps programmatically (reading and writing the same YAML files the skills use), evaluates each step automatically with the same evaluation scripts, runs adversarial agents at key points, and produces a final artifact with scores at every step.

The key insight: **the custom harness does not replace the skills. It composes them.** Everything you have built (skills, EC, evaluation scripts, adversarial agents) becomes the building blocks. The harness is how you put them together into a production workflow.

You will not build one in this program. Building a custom harness is the work of a future Stage 4 Deep Learning Program. What you need as an Embedded Engineer is awareness: what it is, what problem it solves, and when a client team is ready for it.

3:00 – 5:00 PM: THE IIOS STAGE FRAMEWORK

The **Improving Intelligence Operating System (IIOS)** defines stages of AI maturity:

© 2026 Improving

Stage	Name	What It Means	Median Effort
2	Off the Shelf	Using AI tools as-is. Producing with AI but not measuring impact.	n/a
3	Task	Defined tasks with real context. Deliberate RTCC skills, measured improvement, evidence-based iteration, adversarial challenge, bounded agents.	18h
4	Workflow	Chained multi-step workflows with automated harnesses, guardrails, human punch-out points, end-to-end success rates, audit trails.	172h (~10x Stage 3)

DLP Completion is Stage 3. Everything in this bootcamp and the 12-week DLP targets it: personal skills, deliberate invocation decisions, measured iteration, adversarial challenge, bounded agents with a human at every checkpoint.

The Stage 3 ceiling: Stage 3 is personal. Each person's skills reflect their own standards. Review is a human, one piece at a time. To compound, the team must reconcile personal agents into shared workflows and move checking out of human heads and into systems. That is Stage 4.

What Stage 4 requires: team coordination with IT oversight, roughly 10x the effort of Stage 3, and solid Stage 3 foundations. A Stage 4 workflow built on unreliable Stage 3 skills is not faster. It is a slop cannon: errors compound through every connected step. (At 95% per-step accuracy, a 20-step pipeline completes correctly only about 36% of the time.)

Why this matters to you as an Embedded Engineer: client teams will ask "can we automate more of this?" by Week 3 of an engagement. You now have the honest answer: what the prerequisites are (evaluated, iterated, adversarially challenged skills), what the failure modes are at each step of the spectrum, and what the next stage actually costs. You can describe the trajectory without overpromising.

© 2026 Improving

FRIDAY (9:00 AM – 12:00 PM): SYNTHESIS AND ARTIFACTS REVIEW

9:00 – 10:30 AM: THE SPECTRUM REVISITED

Where does each of your Week 1 and Week 2 artifacts sit on the automation spectrum now? For each skill you built:

- Has it earned model invocation (three graduation criteria + EDD prerequisite + invocation test cases), or is there a documented gap?
- Does it have an adversarial agent, and has the evaluator-vs-adversarial delta been examined?
- If placed in a bounded agent chain, which steps upstream and downstream does it depend on, and where would you put the evaluation checkpoints?

Answer these questions in writing. This document is part of your certification gate artifacts.

Then, the coaching deliverable: write a 1-paragraph explanation, for a technically-aware manager, of why each step up the automation spectrum requires evaluation evidence before the next one is safe. Why you would not enable model invocation on an unmeasured skill, and why you would not chain skills into a bounded agent before each one passes its own evaluation. No jargon. This becomes part of your coaching vocabulary, and you will deliver it out loud.

10:30 – 11:30 AM: WEEK 3 ARTIFACTS REVIEW

- [ ] Graduation criteria document for at least one skill (all three criteria answered in writing)
- [ ] Invocation test cases (3 verified positive trigger + 2 quality, plus negative cases) for at least one skill
- [ ] Either a skill with model invocation enabled and test cases committed, or a gap specification document (both valid; vague "not ready" is not)
- [ ] Adversarial prompt for one Week 2 skill, run against 2–3 real outputs, findings documented and classified
- [ ] Evaluator-vs-adversarial comparison document with the delta noted
- [ ] Bounded agent run: full pipeline session output with evaluation and adversarial review at key steps
- [ ] Automation spectrum assessment: written evaluation of Week 1–2 artifacts against the spectrum criteria
- [ ] Manager-audience explanation paragraph: why each spectrum step requires evaluation evidence

11:30 AM – 12:00 PM: PREPARATION FOR WEEK 4

Brief preview of Week 4: the certification gate, the player-coach model, and what the resistance scenarios look like in practice. Candidates should arrive at Week 4 with all artifacts from Weeks 1–3 reviewed and committed.

© 2026 Improving

WEEK 3 DELIVERABLES (COMMITTED TO REPO)

- [ ] Graduation criteria document + invocation test cases for at least one skill
- [ ] Skill with model invocation enabled, or gap specification document
- [ ] Adversarial agent prompt with documented findings (classified: EC gap, skill gap, or false positive)
- [ ] Evaluator-vs-adversarial comparison document
- [ ] Bounded agent run artifact (full pipeline, evaluation and adversarial review at key steps, context window degradation observations noted)
- [ ] Automation spectrum assessment document
- [ ] Manager-audience explanation paragraph

© 2026 Improving

COACHING NOTES FOR INSTRUCTORS

Common failure modes:

- **Candidates enable invocation without negative test cases.** Positive cases prove the skill fires when it should. Only negative cases prove it stays silent when it should not. Over-invocation is the most common and most urgent field failure; require the negative cases before signing off on any flip.
- **Candidates write trigger inputs but never verify them.** "Set up the context, confirm the model actually invokes the skill, check the output meets spec" is the work. If they cannot verify invocation, the trigger conditions are not specific enough yet. The difficulty is the signal.
- **Candidates enable invocation on skills without EDD history.** Check the evaluation history before agreeing. Fewer than 3 documented iteration cycles means the prerequisite is more EDD work, not graduation.
- **Adversarial prompts that duplicate the evaluator.** If the adversarial agent only finds what the evaluation script already catches, it adds nothing. Push on the reviewer role, the mission, and the fact that the agent should never see the EC.
- **Adversarial agents shown the EC.** An agent that knows the criteria grades against the criteria. That is the evaluator's job. The adversarial agent's value is finding what the criteria did not anticipate.
- **Candidates treat the bounded agent run as automation.** It is a manually orchestrated chain with human checkpoints. The moment a candidate describes it as "automated," correct it: the human at each checkpoint is the governance.
- **Candidates want to jump to custom harnesses.** Use the slop cannon line: a Stage 4 workflow on weak Stage 3 foundations compounds errors through every connected step. The 10x effort jump (18h → 172h median) and the team/IT coordination requirements are the honest answer to "why not now."
- **The manager paragraph is taken too lightly.** This is one of the most important coaching deliverables in Week 3. If a candidate cannot explain the evidence-before-automation rationale to a manager without jargon, they cannot advocate for the right approach on a DLP engagement. Require them to actually deliver it out loud.

© 2026 Improving